SOFTWARE ENGINEERING – QUICK REFERENCE (EXAM SURVIVAL)

CHAPTER 1 – OVERVIEW

- Goal: Build high-quality software on time and within budget

- 3 pillars: People – Process – Tools

- Biggest challenge: Balance quality, time, and cost

CHAPTER 2 – SOFTWARE LIFE CYCLE

Waterfall:

- Sequential, fixed requirements

- Main disadvantage: Hard to handle changes

Incremental:

- Develop in parts, integrate gradually

Spiral:

- Risk-driven

- Number of loops depends on risk level

RAD:

- Fast and flexible

- Not suitable for large, complex systems

Agile:

- Development and testing in parallel

CHAPTER 3 – AGILE & SCRUM

Agile values:

- Individuals > Process

- Working software > Documentation

- Customer collaboration > Contract

- Responding to change > Plan

Scrum Team:

- Product Owner

- Scrum Master

- Development Team

(No Project Manager)

Sprint:

- Short, time-boxed iteration

Scrum Master:

- Facilitates process, removes impediments

Definition of Done:

- Agreed criteria to mark work as complete

CHAPTER 4 – PROJECT MANAGEMENT

WBS:

- Most detailed level: Work Package

- Deliverables identified at highest level

- Do not use BOM

Critical Path:

- Longest duration path

- Delay affects whole project

Risk Exposure:

- RE = Probability × Impact

CHAPTER 5 – CONFIGURATION MANAGEMENT

Configuration Item:

- Requirements, design, code, test artifacts

- Emails are not CI

Baseline:

- Official change control point

Versioning:

- major.minor.build.revision

- New feature (compatible): increase minor

Git:

- Distributed, offline work supported

CHAPTER 6 – REQUIREMENTS ENGINEERING

Functional vs Non-functional:

- Functional: What system does

- Non-functional: Performance, security, reliability

Verification vs Validation:

- Verification: Build right product?

- Validation: Meets user needs?

Traceability Matrix:

- Maps stakeholder needs to system features

CHAPTER 7 – SOFTWARE DESIGN

Prototype:

- Low vs High fidelity

- Interface vs Interactive

Design principles:

- Consistency

- Feedback

- Recoverability

- Least Surprise

Good design:

- High cohesion, loose coupling

Architecture:

- Pipe & Filter: Sequential processing

- Layered: Neighbor interaction only

- Repository: Shared data with notification

CHAPTER 8 – SOFTWARE CONSTRUCTION

Programming style:

- Readable, maintainable code

Refactoring:

- Improve structure without changing behavior

Code Review:

- Inspection, Walkthrough

- Early bug detection, knowledge sharing

ROI:

- Return on Investment

- Done in requirement analysis

CHAPTER 9 – SOFTWARE TESTING

Black-box: Input/Output

White-box: Internal logic

Gray-box: Partial internal knowledge

Testing techniques:

- Equivalence → **Reduce test cases**
- Boundary → **Edge values**
- Decision table → **Many conditions**
- State transition → **State-based system**
- Regression → **After change**
- Control flow → **Paths**
- Path testing → **All paths**
- Cause–effect → **Build decision table**